

ZeroCode performance datasheet

Contents

ZeroCode performance datasheet	1
TL;DR	1
Test environment	1
Methodology	2
Per-language datasheet	2
Traffic / saturation analysis	4
Bottlenecks, in priority order	5
Reproduce	6

ZeroCode performance datasheet

Generated by [scripts/benchmark.sh](#) against a live stack. Raw data: [benchmarks/data/languages.csv](#) (per-language) and [benchmarks/data/throughput.csv](#) (saturation sweep). Re-run with `./scripts/benchmark.sh all`.

This sheet answers two questions:

1. **What does each language cost** — CPU, wall time, memory, and compile cost — across light / medium / heavy workloads.
 2. **Where does the server bottleneck under traffic** — measured by ramping concurrency until throughput stops scaling.
-

TL;DR

- **Two hard ceilings gate throughput**, both small and both default-derived:
 1. **Worker execution = 4 concurrent jobs.** The worker runs a `Semaphore(max_parallel)` where `max_parallel` defaults to the CPU count (4 here). Throughput scales linearly to concurrency 4, then collapses.
 2. **API in-flight = 16 requests.** The API's Postgres pool is `max_connections(16)`, and `?wait=true` holds a connection for the whole job. Past ~16 simultaneous in-flight submissions the API returns **HTTP 500 db: pool timed out**.
 - **Peak measured throughput on this 4-vCPU box:** ~37 req/s for a trivial interpreted job, ~3.4 req/s for a cache-missed C++ compile. Both peak at concurrency 4.
 - **The compile cache is the single biggest lever — but only for single-binary compiled languages.** C/Go/Rust/Crystal/... cache the compiled `/box/prog` and drop from a 1–13 s cold compile to ~0.02–0.1 s warm. **JVM and .NET languages do *not* benefit** (Kotlin, Scala, Clojure, Groovy, C#, F# recompile/respawn on *every* submission) and stay expensive (0.7–5.4 s each) under sustained traffic.
 - **Memory is the per-job pressure that matters at concurrency.** Most jobs are <60 MB, but JVM/Pony/compile-heavy peaks hit **113–383 MB** — at 4 concurrent Kotlin compiles that's ~1.5 GB, which matters on small nodes.
-

Test environment

Host	4 vCPU, 15 GiB RAM (shared dev box; ~0.5 GiB free during the run)
Topology	1 API + 1 worker + Postgres 16, <code>deploy/docker-compose.yml</code>
Worker parallelism	<code>ZEROCODE_MAX_PARALLEL unset -></code> <code>available_parallelism() = 4</code>
API DB pool	<code>max_connections(16)</code> <code>(crates/zerocode-api/src/state.rs)</code>
Per-submission limits	<code>cpu <= 30 s, wall <= 60 s, mem <= 1024 MB</code>
Sandbox	<code>NativeSandbox (cgroup v2 + userns + landlock + seccomp + pivot_root)</code>

Numbers are **machine-specific** — they scale with vCPU count (worker parallelism) and the DB pool size. Use the *shape* (where it saturates, the relative per-language costs), not the absolute req/s, for sizing other hardware.

Methodology

Workload. Every language runs the same deterministic kernel — a modular hash loop that is impossible for a compiler to fold into a closed form, so the loop genuinely executes:

```
read N
s = 0
for i in 1..N: s = (s * 1000003 + i) mod 4294967291
print s
```

All intermediates fit in signed 64-bit, so every language prints the **same** value for a given N (used as a correctness check). Programs live in [benchmarks/programs/compute/](#).

Tiers (workload size via stdin N):

Tier	N	Probes
easy	1	startup + (cached) compile + I/O — the fixed per-job floor
medium	1,000,000	runtime / JIT compute
hard	10,000,000	sustained CPU; slow interpreters TLE here

Cold start vs. warm tiers. Before the timed tiers, each language is run once to populate the compile cache; the **cold-start** column is that first run (compile + spawn, measured against an emptied `compile_artifacts` table). The easy/medium/hard columns are **run cost** with the compile cache warm — so the gap between *cold-start* and *easy* is exactly what the compile cache saves.

Esoteric / niche tier. Brainfuck, GolfScript, CJam, Vyxal, Jelly, Samarium, Paradoc, plus Assembly, COBOL, Prolog, SQL, Forth, Verilog, LLVM IR and Pony can't express the scaling loop cleanly, so they're profiled at the easy tier only (a fixed "hello"); medium/hard are **N/A**.

Saturation sweep. Each request carries a globally unique source (so it misses both the result cache and the compile cache -> real sandbox execution), fired at rising concurrency. We record throughput and client-side latency percentiles per level.

Per-language datasheet

cold = first-ever (cold-cache) wall: compile + spawn. **easy/medium/hard** = run wall (s) with compile cached, at N = 1 / 1e6 / 1e7. **hard CPU** = CPU-s at the heavy tier. **mem** = peak RSS (MB) seen across tiers (for compiled languages the cold-compile peak can dominate).

ID	Language	cold (s)	easy (s)	medium (s)	hard (s)	hard CPU (s)	mem (MB)
48	C	0.12	0.023	0.018	0.062	0.047	<1
52	C++	0.87	0.018	0.038	0.072	0.053	<1
60	Go	9.34	0.026	0.031	0.058	0.050	3
62	Java	0.60	0.548	0.559	0.614	0.601	62
63	Node.js	0.17	0.129	0.159	0.346	0.334	16
71	Python	0.05	0.043	0.204	1.573	1.556	3
73	Rust	0.74	0.035	0.032	0.075	0.054	5
100	Bash	0.03	0.025	2.831	27.208	27.128	1
101	Lua	0.03	0.078	0.061	0.167	0.143	<1
102	Perl	0.04	0.039	0.101	0.593	0.576	1
103	Ruby	0.12	0.094	0.162	0.797	0.784	13
104	R	0.30	0.268	0.459	2.053	2.037	56
105	PHP	0.06	0.076	0.052	0.152	0.141	4
106	TypeScript	0.97	1.035	0.962	1.182	1.166	80
110	Fortran	0.17	0.023	0.040	0.103	0.073	<1
111	Pascal	0.17	0.023	0.033	0.074	0.058	1
112	D	2.08	0.027	0.032	0.076	0.054	2
113	Objective-C	0.21	0.021	0.032	0.078	0.053	<1
114	Assembly	0.06	0.057	N/A	N/A	—	5
115	Ada	0.43	0.023	0.035	0.074	0.057	<1
120	Kotlin	5.74	5.281	5.195	5.409	5.385	383
121	Scala	3.88	3.538	3.497	3.575	3.550	181
122	Groovy	0.69	0.743	0.842	0.961	0.940	113
123	Clojure	1.85	2.405	2.112	2.163	2.140	203
130	Haskell	1.14	0.044	0.049	0.094	0.069	11
131	OCaml	0.27	0.035	0.031	0.087	0.066	2
132	Erlang	1.64	1.671	1.739	1.721	0.714	54
133	Elixir	0.49	0.468	0.468	0.590	0.576	53
134	Common Lisp	0.06	0.045	0.058	0.138	0.121	3
140	C#	0.78	0.668	0.684	0.815	0.796	42
141	F#	1.84	1.787	1.719	1.763	1.734	115
150	COBOL	0.26	0.263	N/A	N/A	—	19
151	Prolog	0.08	0.078	N/A	N/A	—	8
152	Swift	1.40	0.035	0.034	0.071	0.058	1
153	Octave	0.21	0.162	2.498	24.354	23.550	9
154	SQL	0.05	0.049	N/A	N/A	—	1
161	Nim	2.83	0.060	0.043	0.111	0.075	1
162	Crystal	13.50	0.032	0.103	0.078	0.050	3
163	Dart	2.02	0.031	0.042	0.103	0.075	9
164	Julia	0.84	0.336	0.554	2.113	2.082	111
170	Racket	0.81	0.558	0.505	0.591	0.563	132
171	Raku	0.51	0.337	1.672	12.252	12.099	94
172	AWK	0.04	0.039	0.139	1.009	0.988	1
173	CoffeeScript	0.48	0.258	0.300	0.484	0.474	24
174	Forth	0.05	0.051	N/A	N/A	—	1
176	Emacs Lisp	0.41	0.166	0.349	2.015	1.994	17
177	Verilog	0.10	0.096	N/A	N/A	—	6
178	LLVM IR	0.28	0.281	N/A	N/A	—	66
179	V	0.28	0.032	0.035	0.076	0.059	1
180	FreeBASIC	0.20	0.024	0.033	0.073	0.058	1
181	PowerShell	0.64	0.622	1.756	8.983	8.882	39
182	Pony	1.15	1.150	N/A	N/A	—	200
300	Brainfuck	0.06	0.061	N/A	N/A	—	4

ID	Language	cold (s)	easy (s)	medium (s)	hard (s)	hard CPU (s)	mem (MB)
301	GolfScript	0.16	0.162	N/A	N/A	—	18
302	CJam	0.26	0.261	N/A	N/A	—	54
303	Vyxal	0.98	0.978	N/A	N/A	—	92
304	Jelly	0.08	0.083	N/A	N/A	—	7
305	Samarium	0.12	0.121	N/A	N/A	—	8
306	Paradoc	0.17	0.174	N/A	N/A	—	19

Reading the table

Compile cache: huge for binaries, useless for JVM/.NET. Compare *cold* to *easy*:

- **Single-binary compiled** (C, C++, Go, Rust, D, Obj-C, Ada, Fortran, Pascal, Haskell, OCaml, Nim, Crystal, Dart, Swift, V, FreeBASIC): cold compile is 0.1–13.5 s but warm runs are **~0.02–0.1 s** — a **100–400x drop**. Crystal (13.5 s -> 0.03 s) and Go (9.3 s -> 0.03 s) are the extreme cases. After the first submission of a given program, these are effectively free.
- **JVM** (Kotlin 5.3 s, Scala 3.5 s, Clojure 2.1 s, Groovy 0.9 s) and **.NET** (C# 0.7 s, F# 1.7 s): *easy* ~ *cold* — the cache does **not** help. Their compile output isn't the single `/box/prog` the content-addressed cache stores, so every submission re-pays compiler + VM startup. **Under traffic these are permanently expensive.**

Slowest at the heavy tier (hard, wall-s): Bash 27.2, Octave 24.4, Raku 12.3, PowerShell 9.0, Kotlin 5.4, Scala 3.6, Julia 2.1, R 2.1, Emacs Lisp 2.0, Python 1.6. Loop-heavy work in shell/array languages is 100–1000x slower than compiled — Bash at N=1e7 burns 27 CPU-s where C burns 0.05.

Heaviest memory (peak RSS, MB): Kotlin 383, Clojure 203, Pony 200, Scala 181, Racket 132, F# 115, Groovy 113, Julia 111, TypeScript 80, LLVM IR 66. Most everything else is <60 MB; the Core-7 compiled langs are <10 MB at runtime.

Heaviest cold compile (s): Crystal 13.5, Go 9.3, Kotlin 5.7, Scala 3.9, Nim 2.8, D 2.1, Dart 2.0, F# 1.8, Swift 1.4, Pony 1.2, Haskell 1.1, C++ 0.9.

Traffic / saturation analysis

Throughput and latency vs. concurrency (unique payloads -> real execution, no cache hits). Two workloads: a trivial interpreted job (Python) and a cache-missed C++ compile.

python-light (interpreted, ~minimal work):

concurrency	throughput req/s	p50 ms	p95 ms	p99 ms	errors
1	10.9	70	88	93	0
2	21.5	71	92	95	0
4	36.7	82	111	125	0
8	20.3	171	245	2118	0
16	2.4	6250	8246	8264	21
32	3.0	6177	16179	16232	56
64	2.8	8144	24492	26451	22

cpp-compile (compile-bound, cache-missed each request):

concurrency	throughput req/s	p50 ms	p95 ms	p99 ms	errors
1	1.1	867	937	988	0
2	2.1	886	1219	1547	0
4	3.4	1118	1222	1288	0
8	3.2	2138	3285	3557	0
16	1.8	8112	10199	10272	9
32	2.8	3085	10497	12137	20

concurrency	throughput req/s	p50 ms	p95 ms	p99 ms	errors
64	4.3	2006	7130	7140	24

What the curve shows

- **Linear scaling to concurrency 4, then the knee.** Throughput rises ~proportionally (1->2->4 ~ 11->21->37 req/s for Python) while latency stays flat — the 4 workers are filling up. At concurrency 4 every worker slot is busy: **this is peak throughput.**
- **Past 4, it gets *worse*, not flat.** At concurrency 8 Python throughput *drops* to 20 req/s and p99 jumps to 2.1 s: 8 CPU-bound jobs are time-slicing 4 cores, so each runs slower and the extras queue. Beyond the worker's parallelism, added load is pure overhead.
- **Errors start exactly at concurrency 16.** That's the API's `max_connections(16)` Postgres pool. With `?wait=true`, each in-flight request holds a pooled connection until its job finishes; the 17th+ concurrent request waits for a free connection and trips the pool's acquire timeout -> **HTTP 500 {"error":"db: pool timed out while waiting for an open connection"}** (confirmed: 50 concurrent requests -> 39x200, 11x500).

Bottlenecks, in priority order

1. **Worker concurrency = `min(CPU count, ZEROCODE_MAX_PARALLEL)` = 4.** The primary throughput ceiling. CPU-bound user code is the workload, so the sandbox is fundamentally CPU-limited: useful parallelism ~ core count. *Fix:* add cores and/or worker replicas; they scale throughput ~linearly (4 -> 37 req/s suggests ~9 req/s per core for trivial jobs). Keep `ZEROCODE_MAX_PARALLEL` ~ vCPU — oversubscribing (the conc-8 row) *reduces* throughput and inflates tail latency.
2. **API Postgres pool = 16, amplified by `wait=true`.** Caps *in-flight* submissions at 16 regardless of worker count; the 17th gets a 500. Because a synchronous `wait=true` request occupies a connection for the job's entire duration, a few slow jobs (a Kotlin compile, a Bash hard loop) can pin the pool. *Fix:* raise `max_connections`; or prefer the **async submit -> SSE/poll** flow (POST returns a token immediately, releasing the connection) for high-concurrency clients instead of `wait=true`.
3. **Per-language cost dispersion (capacity depends on the language mix).** Sustained per-worker rate ~ 1 / wall:
 - Warm compiled (Go/Rust/C, ~0.05 s) -> ~20 jobs/s/worker (~80 across 4).
 - JVM (Kotlin ~5.3 s, *never cached*) -> ~0.19 jobs/s/worker (~0.75 across 4).
 - Cache-missed C++ compile (~0.9 s) -> ~1.1 jobs/s/worker (measured 3.4 at 4). A Kotlin-heavy or compile-cache-missing workload saturates the box ~**100x sooner** than a warm-Go workload. Size for the *slowest expected mix*.
4. **Memory at concurrency.** 4 concurrent Kotlin jobs ~ 4 x 383 MB ~ 1.5 GB; on this ~0.5 GB-free box that risks **sandbox_failure** (observed intermittently on cold JVM startups). Budget `MAX_PARALLEL` x **peak-job-MB** against node RAM, and cap memory-heavy languages with tighter `default_limits`.

Recommended next steps

- For higher throughput: scale **vCPUs + worker replicas** together; set `ZEROCODE_MAX_PARALLEL` to the per-worker core count (don't oversubscribe).
- Raise the API DB pool above 16, **or** steer bursty/high-concurrency traffic to async submit + SSE so connections aren't held during execution.
- Treat the **compile cache as load-bearing** for compiled languages — it's the difference between 9 s and 0.03 s on a Go submission. Investigate extending content-addressed caching to JVM/.NET artifacts (`.jar/.dll`) to remove their per-request recompile.
- For capacity planning, multiply expected RPS by the per-language **wall** from the table above for the languages you actually serve.

Reproduce

```
# Per-language complexity profile -> benchmarks/data/languages.csv
./scripts/benchmark.sh bench
```

```
# Saturation sweep -> benchmarks/data/throughput.csv
./scripts/benchmark.sh sweep
```

```
# Both
```

```
API_BASE=http://localhost:8080 ./scripts/benchmark.sh all
```

For honest cold-compile numbers, clear the compile cache first (`TRUNCATE compile_artifacts;`) and restart the worker so its in-memory cache is dropped, then run `bench`.